

Session 2 — Study Guide: React Context in B4F Hub

This guide explains, in depth, what changed in B4F Hub in Session 2 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Before this session begins, the Session 1 continuity homework (a `NotFound` page for unknown addresses) is reviewed and its official solution is taught, live, at the start of class — if you attempted it yourself first, great; either way, everyone starts today's new content from the same place: `/`, `/community`, `/opportunities`, a working detail page, and a working `NotFound` page.

1. Where the session starts and ends

Start. Everything from Session 1, plus the `NotFound` page.

End. Notifications (toast messages for "post published", "like failed", "applied", etc.) no longer travel through props at all — they go through a `NotificationContext`. A second Context, `SavedOpportunitiesContext`, lets the Navbar show a live count of saved opportunities, built by your team as the teamwork.

2. The problem: prop drilling

A familiar idea first. You already know how to pass data to a component: props. `App` owns the list of notifications and two functions to change it; it hands `onNotify` to `CommunityPage`, which hands it to `CommunitySection`, which finally uses it.

What's wrong with that? Before this session, `CommunityPage.tsx` received `onNotify` as a prop and did exactly one thing with it: hand it straight to `CommunitySection` as a prop of its own.

`CommunityPage` never calls `onNotify` itself. It exists only to hand the prop to the next component.

`OpportunitiesPage.tsx` does the exact same thing. This is **prop drilling**: a value passed through one or more components that never use it themselves, only to reach a distant descendant that does. It's not wrong, exactly — it works — but it means every component in between has to know about a concern that isn't its own, and adding a new page that also needs `onNotify` means wiring the prop through again.

The idea of this whole session: give components a way to reach a shared value directly, without every component in between having to carry it.

3. `createContext` : the shape of a shared channel

TSX

```
interface NotificationContextValue {
  notifications: AppNotification[];
  notify: (message: string, tone: NotificationTone) => void;
  dismissFront: () => void;
}

const NotificationContext = createContext<NotificationContextValue | null>(
  null,
);
```

Line by line. `createContext(defaultValue)` returns a Context object — think of it as a labeled channel. The type parameter (`NotificationContextValue | null`) says what a component will get when it reads from this channel. The default, `null`, is deliberately a value nothing should ever actually use — it exists only so the type checker has something to fall back on before a real Provider exists; we will guard against exactly that case in a moment.

Why decide the shape first? Look at what already exists in `App` before this change: the `notifications` array, and the two functions (`addNotification`, `dismissFrontNotification`). That becomes the Context's value — nothing new is invented, only moved.

4. The Provider: where the state actually lives

TSX

```
function NotificationProvider({ children }: { children: ReactNode }) {
  const [notifications, setNotifications] = useState<AppNotification[]>([]);

  function notify(message: string, tone: NotificationTone) {
    /* ...exactly the old addNotification logic... */
  }

  function dismissFront() {
    /* ...exactly the old dismissFrontNotification logic... */
  }

  return (
    <NotificationContext.Provider
      value={{ notifications, notify, dismissFront }}
    >
      {children}
    </NotificationContext.Provider>
  );
}
```

Line by line. `NotificationProvider` is an ordinary component with one job: own the state, and make it available via `<NotificationContext.Provider value={...}>`. Everything rendered as `{children}` — no matter how deeply nested — can now read that `value`.

Where does the Provider go? Wrapped around `<App />` in `main.tsx`, right next to `BrowserRouter`:

TSX

```
<BrowserRouter>
  <NotificationProvider>
    <App />
  </NotificationProvider>
</BrowserRouter>
```

Why does this feel familiar? Because it's the exact same idea as `BrowserRouter` from Session 1: wrap once, near the top, and every descendant can use what it provides. `BrowserRouter` happens to use Context internally too — you were already using this pattern without the name.

5. `useContext` and the custom hook

TSX

```
function useNotify() {
  const context = useContext(NotificationContext);

  if (!context) {
    throw new Error("useNotify must be used inside a NotificationProvider");
  }

  return context;
}
```

Line by line. `useContext(NotificationContext)` reads the nearest matching Provider's value — `null` if there is none above the caller. Wrapping it in `useNotify()` centralizes the null-check once, in one place, instead of every component that wants to read notifications repeating it (or worse, skipping it and crashing somewhere confusing later).

Using it is one line:

TSX

```
function ToastQueue() {
  const { notifications, dismissFront } = useNotify();
  /* ...renders exactly as before... */
}
```

No props. `ToastQueue` reaches directly into the Context.

6. What disappears from `App.tsx`

Once every consumer (`ToastQueue`, `CommunitySection`, `OpportunitiesSection`, `OpportunityDetailPage`) reads from `useNotify()` instead of receiving a prop, `App` has nothing left to do with notifications: the `useState`, the two functions, and every `onNotify={...}` prop on every route — all deleted. `App` shrinks back to being just layout and routing, which is all it should have ever needed to be.

7. Where Context does NOT belong

Not everything should move to Context. Inside `OpportunitiesSection`, `searchText`, `typeFilter`, `workModeFilter`, and `applyingId` all stayed exactly as local `useState` — nothing outside this one component needs them. Moving them to Context would add a file, a Provider, and a hook for zero benefit, and would make the component's own behavior harder to trace (its state would no longer be visible just by reading the component itself).

The rule of thumb: reach for Context when a value is needed by a component with no direct ancestry relationship to whoever owns it (or by many components at very different depths). Keep state local when only that one component — or its own direct children via ordinary props — needs it.

8. Your teamwork: `SavedOpportunitiesContext` (a sibling problem)

Before the teamwork, `savedIds` (which opportunities are saved) lived only inside `OpportunitiesSection`. The task: let the `Navbar` show a live count of saved opportunities. The `Navbar` is not a parent, child, or any other kind of descendant of `OpportunitiesSection` — they are **siblings**, both direct children of `App`. No prop chain could carry `savedIds` from one to the other without first lifting it all the way up to `App` and passing it back down to both — a bigger, worse redesign just to satisfy this one need. This is exactly the case Context exists for:

TSX

```
const { savedIds } = useSavedOpportunities();
```

called directly inside `Navbar`, with zero props involved.

9. FLEX: when NOT to reach for Context

`isSaved` / `onToggleSaved` travel from `OpportunitiesSection` through `OpportunityList` to `OpportunityCard` — two hops, all inside the same feature, all components that exist *because of* opportunities. Converting this to Context would add a file, a Provider, and a hook to save nothing: the props are short, clearly named, and every component in the chain already belongs to the same concern. **Context is not automatically "better" than props** — use the simplest tool that reaches the actual distance a value needs to travel.

Glossary

Word	Meaning
Prop drilling	Passing a prop through components that don't use it, only to reach one that does.

Word	Meaning
Context	A channel, created by <code>createContext()</code> , that lets a value skip prop drilling.
Provider	The component that supplies a Context's actual value to everything rendered inside it.
Consumer	Any component that reads a Context's value — in this course, always via a custom hook.

Review questions

1. What is prop drilling? Point to the exact line in `CommunityPage.tsx` (before this session) that shows it.
2. What are the three pieces of the Context pattern used in this course, and what does each do?
3. Why does `useNotify()` throw an error instead of returning `null` quietly?
4. Name one piece of state that correctly stayed local this session, and explain why.
5. Why couldn't a prop solve the Navbar's saved-count badge without a bigger redesign?

Answers

1. `CommunityPage` receives `onNotify` and does nothing with it except hand it to `CommunitySection`:

```
function CommunityPage({ onNotify }: CommunityPageProps) { return <CommunitySection
  onNotify={onNotify} />; }
```
2. The Context object (the channel, created by `createContext`, never exported directly); the Provider component (owns the real state, supplies it via `.Provider value={...}`); the custom `useX()` hook (wraps `useContext`, throws a clear error if there's no Provider above).
3. A thrown error surfaces immediately, at the exact call site, with a message that says exactly what's wrong. Silently returning `null` would push the bug downstream to wherever the value is actually used — a far more confusing crash, or no crash at all, just broken behavior.
4. For example `searchText` in `OpportunitiesSection` — nothing outside that one component needs it, so moving it to Context would add complexity for no benefit.
5. `Navbar` and `OpportunitiesSection` are siblings under `App`, not parent and child — a prop could only reach from one to the other by first being lifted all the way up to `App` and passed back down to both, which is itself the drilling problem, just moved one level higher.

What you should be able to explain now

- ☐ What prop drilling is, using a real example from this app.
- ☐ What `createContext`, a Provider, and `useContext` each do.
- ☐ Why this course wraps `useContext` in a small custom hook.
- ☐ At least one real example of state that correctly stayed local, and why.
- ☐ At least one real example where Context solved something props could not.

Not yet

`useReducer`, Redux/Redux Toolkit, splitting Context by concern for performance (state/dispatch contexts), Next.js. These come later; do not use them even if you already know them from elsewhere.